

SHARP: A Smart Hostel Administration and Resource Platform

Digital Workflows for Leave, Mess, Allotment, Maintenance and Laundry Services

Project Proposal for UCS503

Submitted to: Ms. Anushka

Thapar Institute of Engineering and Technology

Vaibhav Kansal, CSED, Roll No: 1024160097

Ali Ekram, CSED, Roll No: 1024160138

Riya Gupta, CSED, Roll No: 1024160122

Ayush Kamboj, CSED, Roll No: 1024160118

September 15, 2026

1 Higher order goal

This project aims to improve the accountability and traceability of everyday hostel administration by replacing handwritten applications, physical registers and loosely tracked complaints with a single auditable digital record. While the underlying goal is broader institutional efficiency, the immediate engineering objective is to translate *reliable, traceable service delivery* across five hostel processes into a measurable, deployable software solution.

2 Time to value

- We will deliver meaningful increments quickly by building authentication and role management first, then layering the five service modules on top so a usable core exists early.
- We will prioritize fast feedback by phasing delivery (leave, complaints, mess, allotment, laundry, then notifications and deployment) so a working minimum viable product exists even if later phases are unfinished.
- Success is defined by what can be delivered and measured in the first iteration (leave approval and complaint tracking), then improved based on warden and student feedback.

3 Problem Statement

Several routine hostel processes are partially manual or lack tracking. Five pain points motivate the system:

- **Leave and passes:** handwritten applications and manual parent contact create paperwork and delay.
- **Mess feedback:** register-based collection discourages honesty because identity is visible to mess staff.

- **Allotment:** when cluster members pay at different times, a partly reserved room may be taken by others, breaking clusters.
- **Maintenance:** complaints stay unresolved for days, progress is untrackable and preferred repair slots go unhonoured.
- **Laundry:** lost or damaged items are reported informally if at all, so recurring problems (a particular day, batch or item type) go unnoticed.

Why this matters now (contextual relevance): With a hostel population large enough that staff bandwidth is limited, even small tracking improvements (a unique complaint ID, an expiry rule, a Service Level Agreement (SLA) timer) can noticeably reduce unresolved cases and disputes.

4 Proposed Solution

4.1 Overview

Build a **web-based** hostel administration platform with role-based workflows for students, parents, wardens, caretakers, mess staff and laundry staff, comprising five modules:

- **Visitor and leave management:** day-out, regular and emergency requests routed as Student → Parent verification → Warden approval → digital pass with a unique ID.
- **Mess management:** categorised feedback on quality, hygiene, quantity, menu, timing and service, plus special meal requests; Feedback remains anonymous to mess staff, while internal records help prevent misuse
- **Allotment:** a temporary room reservation for a cluster leader that expires after a set window (e.g. 48 hours) if payment is incomplete, preserving clusters without weakening the fee requirement.
- **Complaint/maintenance management:** an ID and priority per report, caretaker assignment, and closure only after student verification, with Service Level Agreement (SLA) escalation to the warden.
- **Laundry issue tracking:** reports of missing, damaged, wrong or stained items, or delays, logged against each submission for later aggregation.

4.2 Core workflow (complaint lifecycle example)

1. Student files a complaint (category, preferred slot) and receives a complaint ID with an assigned priority.
2. A caretaker is assigned and either agrees the proposed slot or offers an alternative; a Service Level Agreement (SLA) timer runs independently and notifies the warden if breached.
3. The caretaker completes the repair and the student verifies resolution.
4. If verified, the complaint closes; if not, it reopens against the same ID.

The following figure illustrates the complete lifecycle of a maintenance complaint, from submission and caretaker assignment to resolution, verification, and closure.

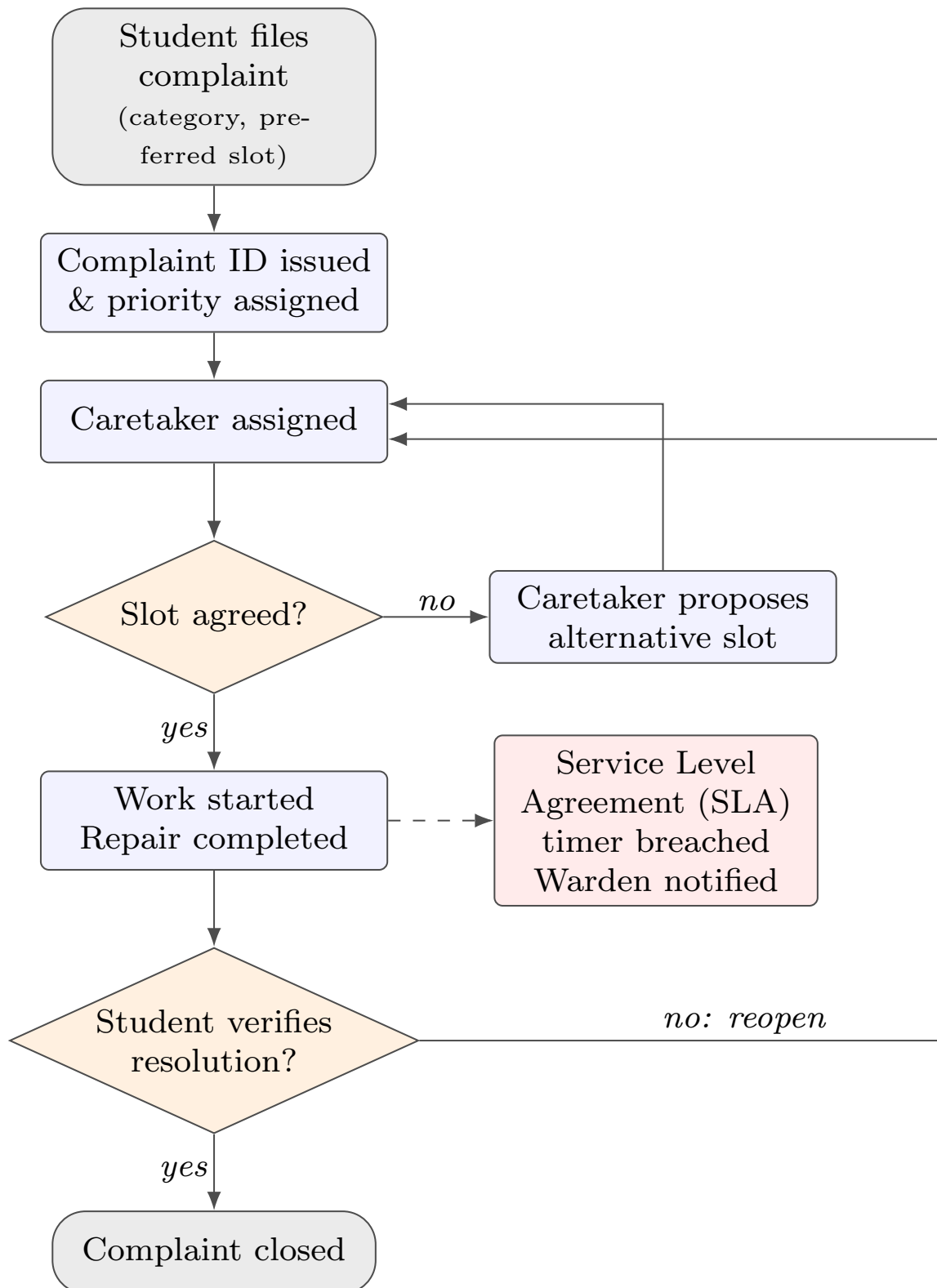


Figure 1: Figure 1: Maintenance Complaint Lifecycle.

4.3 Operational constraints for an educational, campus setting

- Every sensitive endpoint checks both identity and permission, so a student cannot reach administrative dashboards, parent records or the identity behind anonymous feedback.
- Modules stay separable, so one module can change without redesigning the whole application.
- Design for deployability using common web hosting and managed databases.

5 Solution Approach

We focus on layered architecture, role-based access control and workflow correctness rather than novel algorithm research.

5.1 Layered architecture

A layered architecture separates concerns across five tiers:

- **Interface:** role-specific portals for students/parents, wardens/admins, and caretaker/mess/laundry staff apps.
- **Application:** REST API endpoints, authentication (hashed credentials, tokens), and role-based access control.
- **Application Services:** the five modules (leave & pass, mess feedback, allotment, complaints & Service Level Agreement (SLA), laundry issues) plus notifications and analytics.
- **Data:** users, requests, complaints and reports; status history and audit records; backup and recovery.
- **External services (optional):** email, SMS or push notifications.

5.2 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features across the five modules.

6 Evaluation Criterion

We define success using measurable metrics tied to the core workflows.

6.1 Primary evaluation metrics

- **Complaint resolution time:** time between complaint filing and verified closure.
- **Service Level Agreement (SLA) compliance:** share of complaints closed within the defined SLA window.
- **Attribution:** measured from database timestamps (filing, assignment, completion, verification).

6.2 Secondary evaluation metrics

- **Paper Reduction:** Reduction in paper-based leave applications following the implementation of the system.
- **Leave approval time:** average time from student request to digital pass issuance.
- **Reopen count:** number of complaints reopened after a failed verification.
- **Laundry issues reported per month** and recurring-issue detection rate.
- **Reservation conflicts avoided:** clusters preserved through the expiry-based allotment mechanism.

6.3 Pilot validation plan

- Recruit a pilot hostel wing with student, warden, caretaker, mess and laundry staff roles.
- Compare metrics before/after within the iteration window by logging baseline estimates from existing manual records.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in the number of students, requests and concurrent dashboard usage by:
 - decoupling the interface, application and business-logic layers,
 - keeping modules separable so one can change without redesigning the rest,
 - designing the schema to admit further hostels without redesign.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - a managed database with backup and recovery,
 - token-based authentication and role-based access control,
 - a CI/CD pipeline for staging/production deployment.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project. The layers are specified by responsibility, not by product, so the implementation stack will be finalised at build time once hosting, library maturity and team familiarity are weighed:

- Web framework for REST APIs and role-specific reactive pages (e.g. React with Tailwind CSS).
- Managed backend runtime and database (e.g. Node.js with Express and MongoDB).
- Token-based authentication with hashed credentials.
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

- Optional notification services (email, SMS, push).

No workflow rule, role definition or metric depends on a particular language, so deferring the final stack choice costs nothing.

9 Project scope and deliverables

9.1 Initial deliverable

- Authentication and role-based access control for all six roles
- Leave and visitor pass workflow: Student, Parent, and Warden
- Complaint lifecycle: filing, assignment, SLA timer, verified closure
- Basic logging and timestamps for resolution-time measurement
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent deliverables

- Mess feedback module with anonymity and abuse-deterrence records
- Allotment module with temporary reservation and expiry
- Laundry issue tracking with aggregated recurring-issue reports
- Notification layer (email/SMS optional; can start in-app)
- Admin/warden analytics dashboard

10 Risk Management

- **Unauthorised access:** mitigated by role-based authentication.
- **Forged parent approval:** mitigated by secure tokens plus warden verification.
- **Abuse of anonymous mess feedback:** mitigated by rate limits and internal audit records.
- **Premature loss of a room reservation:** mitigated by expiry rules and transactional validation.
- **Disputed laundry loss:** mitigated by timestamped reports tied to each submission.

11 Summary

This project proposes SHARP, a web-based hostel administration platform that brings five hostel processes—leave and passes, mess feedback, room allotment, maintenance complaints, and laundry—into one system. The project will implement role-based access, digital workflows, complaint tracking, verification, notifications, and basic logging. The initial implementation will focus on authentication, leave and visitor pass management, and the maintenance complaint lifecycle, followed by the remaining modules. The aim is to make hostel processes easier to manage, track, and verify through a single digital platform.